

Design rationale for REQ1:

With REQ1, most of the new Actors are just copy paste of previous Actors, with their values changed for the new requirements, for “Eldentree Gaurdian” and “LivingBranch”. To make them survive on the void, the Ability enum is added with the GOD_ON_VOID enum. To make the LivingBranch immovable, the Wander Behaviour is deleted from the behavior priority list. For the new spawners (Bushes, Huts), a Monster Factory list is added so that the spawner can create multiple Actors at once, rather than one in the previous design. This simply iterates through the list and spawns them (with getChance() and so on). For Gates with multiple destinations, it works very similarly to the new spawners. An array is passed in that denotes all the maps to teleport to via the Action list. This makes the engine search through the multiple maps in the Gate and subsequently make n-amount of Actions to give the user the ability to teleport to those different maps. Having the array passed into the spawners and gates allows for easy extensions to be made (e.g. the Gate can have 10 maps to teleport to, and multiple more Actors in the spawners). The new map, the Overgrown Sanctuary map is also copied and pasted from the previous map and changed the map name and the contents of the new map are from the requirements.

Design Rationale for REQ2:

This is a simple task where you just create a Blacksmith then if an actor is around the Blacksmith then it will have the ability to upgrade items. To do that, did made an interface called Item Upgrade (which is new to A3) which includes 2 main functions Return Upgrading to call whenever the item needs upgrading and it will execute that. This function could be overridden in many other classes of items that can be upgraded like Flask, Heal, Broadsword, and Great Knife... so I did modify those classes from A2 to satisfy the requirements of A3. For example in the class Great Knife, I can override the Return Upgrading and then increase the hit rate by 1 percent if the player has enough Runes to upgrade it. Also, another change that I have made to the previous code for Assignment 3 is that I add the status CAN_NOT_UPGRADED to the Item Status, and in the allowable actions of the blacksmith I check if the item does not have that enum then activate the upgrade action. And I just put that enum for the giant hammer as that cannot be upgraded.

Design Rationale for REQ3:

As the player awaits their weapons and items upgrades to complete, the blacksmith decides to share their story and experience on the land of the lords with the player. For that, we made the Conversation interface which contains 2 functions which are Mono louge String which asks for the options of the player and then returns the message, and the String List which generates an array list of the message and also processes the conditions, like if the player holds the great knife, it will return a message, if the player did not defeat the boss then it will print a different message. I did override those in the Black Smith class (this is a new class not in A2), I have added into the allowable actions the Conversation action which has the execute function and it will return the message once it is called. This code could be extended for example if there are other messages to return in any other condition, I can just add the Conversation action and then process the message condition by the String List function. And the code for REQ4 is also extended from this as in the class Traveler, I also reused those functions of the Conversation interface.

Design rationale for REQ4:

So for task 4 I just extended what I did in task 3, I also used the interface Conversation with 2 functions which are String List to take into account the condition, when to generate which message, and the function mono lounge string to generate the options. So basically the fact that I used the array list for the String list function enables us to have dynamic messages for each actor like Traveler or Black Smith. If you want to extend the code, you could just simply use the String list functions for dynamic options. That also increases the code reusability and I think my code is easier to maintain as that function in the Conversation interface could be reused many times and in many situations, as it can deal with dynamic elements.

Design rationale for REQ5:

In regards to this task, I did refactor the unconscious function in the Player class to make the player able to wake right back after getting knocked out unconsciously but still remain the attribute of that player. For example, if the player has already consumed the blood berry, the maximum health of that player after awake will still be 155 instead of 150. I also refactored the Fancy Message class, just adding the A DREAM message to print whenever the player got knocked out unconsciously. Besides, I also added an enum for the boss, in order to check if it is not boss or player then just remove out from the map. I do not really know how this code can be extended but it actually mostly extended from our previous code, which is from the Player class, the Fancy message and just adding a new player to the world in the application. The last requirement of REQ5 involves scanning the entire map for Actors, Grounds, and Items. This uses an Iterator of the map using GetXRange() and GetYRange(), and an order n^2 for loop for the x iter and y iter. During the loop, it scans the entire playfield, using the Ground and Location methods for object detection. When Actors are detected, the Actor is checked if it is neither a BOSS nor a PLAYER. BOSS and PLAYER are detected using the Ability enum. This means that all spawned enemies from a Factory (Bush, Graveyard, and so on) are deleted using the unconscious() method. With the Grounds (Gates), the loop checks if it is a DoubleGate or a BurialGroundGate (using instance), and then it downcasts to the class to use the setGetStatus() method to change the Gate from Unlocked to Locked. For Items (Runes), again instance is used. Then, the Runes get removed from the map.